Chapter 5 Assignment

Highly-articulated Legged Robots

Zihao Liu
Student ID: 220261979
213222647@seu.edu.cn

October 2026

Abstract

This report covers Chapter 5 of the course. Part I comments on the two textbook examples: the ZMP planner (Example 5.1) and the LittleDog gait optimization (Example 5.2). The supplied notebooks were executed, the results are interpreted with the centroidal-dynamics and ZMP theory of the chapter, and the sensitivity to footstep timing, disturbances and gait parameters is studied. Part II solves all exercises of Section 5.8: footstep planning via mixed-integer quadratic programming (MIQP), the conceptual exercise on the zero-moment point, and footstep planning via a graph of convex sets (GCS). Both programming exercises pass their autograders with 16/16 and 16/16. The complete Python code and the executed notebooks accompany the submission; the most important code is reproduced in the appendix.

Contents

1 Overview and Reproducibility **2**

2 Example 5.1: The ZMP Planner **3**
2.1 Model: the linear inverted pendulum and the ZMP . . . 3
2.2 Nominal plan and timing sensitivity . . . 4
2.3 The feedback policy rejects disturbances . . . 6

3 Example 5.2: LittleDog Gait Optimization **7**
3.1 Model and decision variables . . . 7
3.2 Constraints and costs . . . 7
3.3 Results . . . 8

4 Section 5.8(a): Footstep Planning via MIQP **11**
4.1 Terrain and decision variables . . . 11
4.2 The four constraints and the objective . . . 11
4.3 Results on the two benchmark terrains . . . 12
4.4 Step-span sensitivity and route switching . . . 13

5 Section 5.8(b): Understanding the Zero-Moment Point **14**
5.1 Free-body diagrams . . . 14
5.2 (c) The ZMP without an ankle actuator . . . 15

5.3 (e)–(f) With an ideal ankle actuator 16
5.4 (g)–(h) Foot design 16

6 Section 5.8(c): Footstep Planning via GCS **17**
6.1 Transcription 17
6.2 Results 18
6.3 MIQP versus GCS 19

7 Conclusions **20**

8 Code and Reproduction **20**

A Key Code Snippets **21**
A.1 ZMP planner evaluation 21
A.2 MIQP constraints 22
A.3 GCS edges 22
A.4 LittleDog centroidal constraints (sketch) 23

1 Overview and Reproducibility

The assignment consists of:

- **Examples** (code supplied; run and comment): Example 5.1 (`ZmpPlanner`) and Example 5.2 (LittleDog gait optimization).
- **Section 5.8** (all exercises): MIQP footstep planning, the zero-moment-point exercise, and GCS footstep planning.

All simulations use Drake 1.58.0 and the `underactuated` package 2026.9.8 in `/Users/leo/venvs/drake/bin/python` (NumPy 2.5.4, SciPy 1.18.1, Matplotlib 3.11.2). SNOPT is available in this Drake build and is used both for the LittleDog trajectory optimization and for the GCS shortest-path convex relaxation. The exercise notebooks were executed headlessly with `COLAB_TESTING=1`, which prevents interactive visualizers from blocking. The LittleDog notebook was executed *without* that flag, because `running_as_notebook` must be true for SNOPT to be allowed its full iteration budget; the complete optimization takes 8.3 s.

Contents	Location
Example notebooks	`Ch-5/Ch.5-examples/Exp5'1-*.ipynb`, `Exp5'2-*.ipynb`
Section 5.8 notebooks	`Exp5'8-footstep-planning.ipynb`, `Exp5'8-footstep-planning-gcs.ipynb`
Analysis scripts and figures	`Ch-5/exp5_analysis/`
This report	`Ch-5/Ch5_Assignment_Report/`

The autograder results are

```
MIQP footstep planning:  16/16.    GCS footstep planning:  16/16.
```

How to read this report

The report has three layers: *models*, *optimization* and *interpretation*. If an equation looks dense, read the **Intuition** boxes first: they give the physical picture without algebra. The **Recipe** boxes summarise a reusable procedure, and the **Common pitfall** boxes point out the mistakes that are easy to make.

Part	What is done	What to look for
Example 5.1	LIPM + `ZmpPlanner`	CoM lags the ZMP; timing changes sway; feedback rejects pushes
Example 5.2	LittleDog trajectory optimization	contact schedule; CoM and angular-momentum constraints; gait snapshots
Section 5.8 (MIQP)	Big-M footstep planning	one stone per foot; step-span limit; lateral detour
Section 5.8 (ZMP)	Foot/leg/bob analysis	ZMP line of action; heel lift; actuated posture range
Section 5.8 (GCS)	Shortest path in a graph of convex sets	copies of stones; edge constraints; LP relaxation

The big picture in five sentences

A highly articulated robot that stands on flat ground can be planned from the outside in: first choose footsteps, then choose a CoM trajectory whose zero-moment point stays inside the support polygon, and finally realize that CoM trajectory with the whole body. The CoM equations are exact for the full multibody system; the only approximation is the set of contact forces the robot can produce. Footstep planning is a combinatorial problem, solved either by branch and bound over binaries (MIQP) or by a shortest path in a graph of convex sets (GCS). The ZMP is the bridge between the two layers: it turns a contact constraint into a constraint on CoM acceleration.

Symbol	Meaning
$\mathbf{p}_{cm} = (x, y, z)$	center of mass position
$\mathbf{p}_{cp}$, $\mathbf{p}_{zmp}$	center of pressure / zero-moment point
m	total mass
z	constant CoM height in the LIPM
$\mathbf{f}_i$	contact force at contact point i
μ	friction coefficient
d	step span (side of the reachable square)
$\delta_{t,i}$	binary: foot at step t is on stone i
N, L_1, L_2, h	exercise parameters (leg length, toe/heel offsets, ankle height)

2 Example 5.1: The ZMP Planner

2.1 Model: the linear inverted pendulum and the ZMP

For a point mass at constant height z, the horizontal CoM dynamics are the linear inverted pendulum model (LIPM)

$$\ddot{\mathbf{c}} = \frac{g}{z}\left(\mathbf{c} - \mathbf{y}\right), \tag{1}$$

where $\mathbf{c} = (x, y)$ is the CoM position and $\mathbf{y}$ is the zero-moment point. On flat ground the ZMP coincides with the center of pressure (CoP), i.e. with the point where the resultant ground reaction acts. Equation (1) is another way of writing the moment balance

$$\mathbf{y} = \mathbf{c} - \frac{z}{g}\ddot{\mathbf{c}}. \tag{2}$$

The ZMP must lie in the support polygon; for a single flat foot this is the segment between heel and toe.

Drake's `ZmpPlanner` takes a desired ZMP trajectory $\mathbf{y}_d(t)$ (a piecewise polynomial), an initial CoM state $\mathbf{x}_0 = [\mathbf{c}_0; \dot{\mathbf{c}}_0]$ and the height z, and solves

$$\min_{\mathbf{y},\ddot{\mathbf{c}}} \int_0^{t_f} \left[\|\mathbf{y}(t) - \mathbf{y}_d(t)\|^2_{\mathbf{Q}_y} + \|\ddot{\mathbf{c}}(t)\|^2_{\mathbf{R}} \right] dt + \bar{\mathbf{x}}(t_f)^\top \mathbf{S}\, \bar{\mathbf{x}}(t_f), \tag{3}$$

subject to (2), where $\bar{\mathbf{x}}$ is the state relative to the final ZMP and $\mathbf{S}$ is the infinite-horizon LQR cost-to-go of the same dynamics. The solution is a nominal CoM trajectory together with a time-varying linear feedback policy

$$\ddot{\mathbf{c}}^* = \mathbf{K}(t)\left(\mathbf{x}(t) - \mathbf{x}_{\text{nom}}(t)\right) + \ddot{\mathbf{c}}_{\text{nom}}(t). \tag{4}$$

This is the important practical point: the planner does not only return a trajectory, it returns a policy that rejects state perturbations.

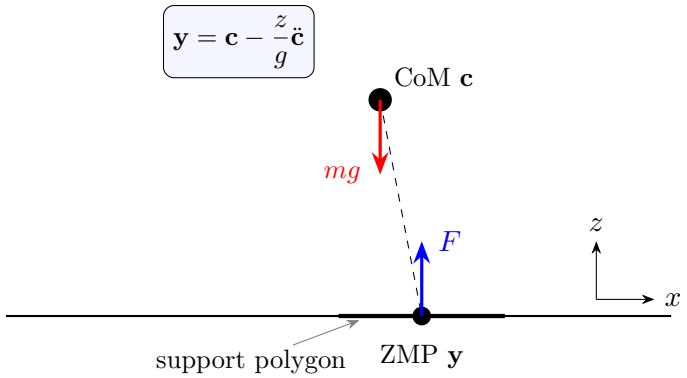


Figure 1: The LIPM and the ZMP. The ground reaction at the ZMP and gravity at the CoM form a couple that produces the CoM acceleration. Keeping the ZMP inside the support polygon (the black segment under the foot) keeps the foot flat.

Building a ZMP plan from footsteps

1. Choose a footstep sequence $\mathbf{p}_0, \mathbf{p}_1, \ldots$ and a timing (single-support and double-support durations).
2. Build the desired ZMP as a piecewise-linear path between the foot centers: hold at each foot during single support, interpolate during double support.
3. Call `ZmpPlanner.Plan(zmp_traj, x0, height)`.
4. Evaluate `get_nominal_com`, `get_nominal_comd` and `ComputeOptimalCoMdd`; use the latter as the feedback policy.
5. Check that the realized ZMP $\mathbf{c} - (z/g)\ddot{\mathbf{c}}$ stays inside the support polygon and close to the desired one.

2.2 Nominal plan and timing sensitivity

The supplied notebook was completed with a plotting and evaluation block. The footsteps are

$$\text{footsteps} = [(0, -0.05), (0.5, 0.05), (1, -0.05), (1.5, 0.05), (2, -0.05), (2.5, 0.05)],$$

i.e. a 0.5 m stride with a 0.1 m lateral sway, and the CoM height is $z = 1$ m. The nominal timing is a 0.5 s double support and a 1.0 s single support.

Table 1: Example 5.1: nominal plan metrics.

Quantity	Value
Plan duration	8.50 s
Peak lateral CoM excursion $\max_t \lvert\text{CoM}_y\rvert$	46.3 mm
Maximum ZMP tracking error	0.158 m
RMS ZMP tracking error	0.078 m
Final CoM x	2.490 m (last footstep at 2.500 m)

The CoM sway is only 4.6 cm although the footsteps alternate by 10 cm; the finite Q/R weighting

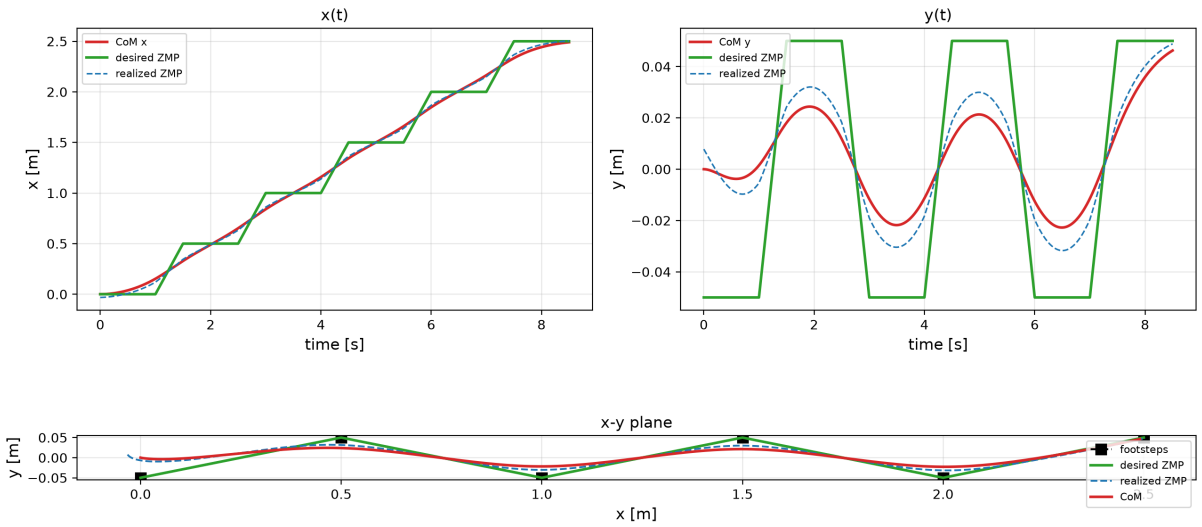


Figure 2: Example 5.1, nominal plan. Top: CoM and realized/desired ZMP in $x(t)$ and $y(t)$. Bottom: top view. The realized ZMP is smoother than the commanded staircase because the quadratic cost (3) trades ZMP tracking against CoM acceleration.

makes the CoM cut the corners of the ZMP staircase. The ZMP tracking error is not zero: with $\mathbf{Q}_y = I$ and $\mathbf{R} = 0.1I$ the planner prefers a smooth CoM acceleration and accepts a lag of up to 15.8 cm during double support. This is a useful reminder that a "ZMP plan" is an optimal-trade-off plan, not a hard equality constraint.

Table 2: Example 5.1: footstep-timing sensitivity (DS = double support, SS = single support).

DS [s]	SS [s]	duration [s]	peak $\lvert\mathrm{CoM}_y\rvert$ [m]	max ZMP error [m]
0.2	1.4	9.40	0.0490	0.204
0.5	1.0	8.50	0.0463	0.158
1.0	0.5	8.00	0.0411	0.095
1.0	1.0	11.00	0.0485	0.101

Comments. The single-support duration is the time the ZMP is held at one foot; the double-support duration is the time it travels between feet. A long single support leaves the CoM on its own for longer, so the pendulum falls farther laterally and the planner must use a larger sway. A long double support smooths the ZMP motion and reduces the tracking error, but it makes the plan longer. The fastest plan in the table (DS = 1.0, SS = 0.5) is also the one with the smallest tracking error, because the ZMP moves quickly between feet and then rests, which is easy for the LIPM to follow.

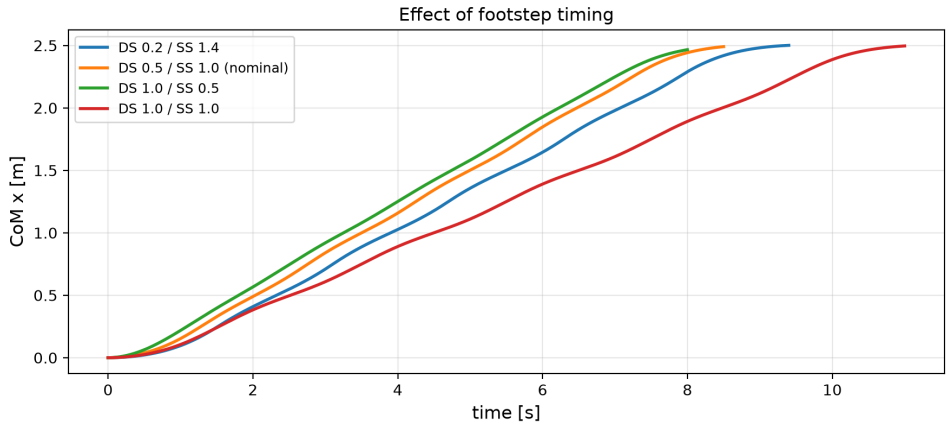


Figure 3: Example 5.1: CoM $x(t)$ for four footstep timings. Longer single support means a longer plan and larger lateral sway; a long double support with a short single support gives the fastest walk in this set.

2.3 The feedback policy rejects disturbances

The planner's value function gives the policy (4). We simulated the LIPM in closed loop from four initial states: the nominal state, a +0.30 m/s and a −0.30 m/s horizontal velocity perturbation, and a $(5, 3)$ cm CoM position offset.

Table 3: Example 5.1: closed-loop response to initial-state perturbations.

Case	max x deviation [m]	final x deviation [m]	final y deviation [m]
nominal	0	0	0
push +0.30 m/s in x	0.0709	1.2×10^{-6}	0
push −0.30 m/s in x	0.0709	-1.2×10^{-6}	0
CoM offset $(5, 3)$ cm	0.0500	8.5×10^{-8}	5.1×10^{-8}

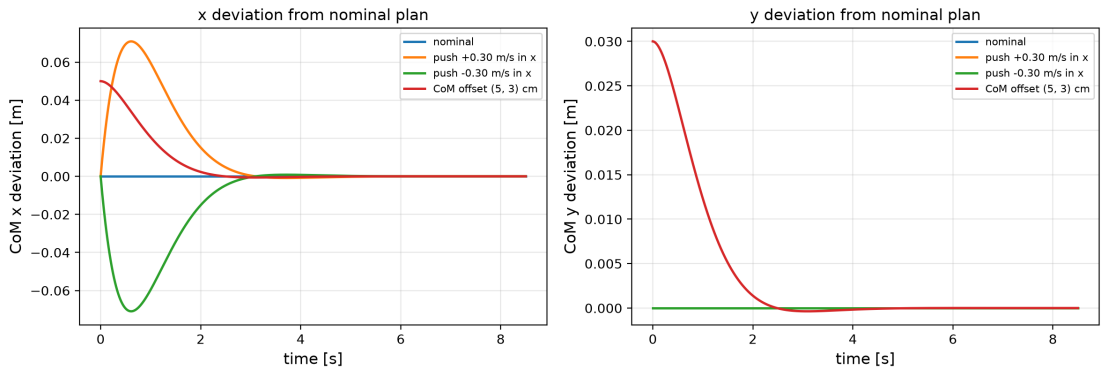


Figure 4: Example 5.1: deviation from the nominal plan under initial-state perturbations. The feedback policy removes a 0.30 m/s push within the 8.5 s plan; the residual at the final time is at the level of integration error.

Why a ZMP planner is a feedback controller

The LIPM is an unstable inverted pendulum: a small velocity error grows like $\exp(t\sqrt{g/z})$. The ZMP plan alone would diverge. The value function of the LQR problem, however, defines the optimal cost-to-go for every state, and its gradient gives a linear feedback law. Pushing the CoM forward makes the policy move the ZMP forward, which decelerates the CoM. In this sense the ZMP trajectory is a feedforward term and the policy is the feedback term.

The ZMP is not the CoM

A common mistake is to draw the CoM path and call it the ZMP, or to assume the ZMP must equal the CoM projection. They coincide only when the CoM acceleration is zero. During walking the CoM is constantly accelerating, and the ZMP leads or lags it by $(z/g)\ddot{\mathbf{c}}$. In Fig. 2 the desired ZMP is a staircase while the CoM is a smooth curve; the horizontal gap between them is exactly the acceleration term.

Comments. The example makes the layering of Chapter 5 concrete: the footstep sequence defines a ZMP trajectory, the ZMP trajectory defines a CoM plan and a feedback policy, and a whole-body controller would then realize the CoM plan with the joints. The LIPM ignores the angular momentum about the CoM; for slow walking this is a good approximation, but fast motions or arm swing require the centroidal-dynamics formulation discussed in the chapter.

3 Example 5.2: LittleDog Gait Optimization

3.1 Model and decision variables

LittleDog is a small quadruped with a floating base and three joints per leg (hip roll, hip pitch, knee): $n_q = 7 + 12 = 19$ positions and $n_v = 6 + 12 = 18$ velocities. The optimization is a direct transcription with $N = 21$ knot points. The decision variables are

- the $N - 1$ time steps $h_n > 0$;
- the configurations $\mathbf{q}_n \in \mathbb{R}^{19}$ and velocities $\mathbf{v}_n \in \mathbb{R}^{18}$;
- the four contact forces $\mathbf{f}_{i,n} \in \mathbb{R}^3$ at the four feet;
- the CoM position/velocity/acceleration $\mathbf{c}_n, \dot{\mathbf{c}}_n, \ddot{\mathbf{c}}_n$;
- the centroidal angular momentum $\mathbf{H}_n$ and its rate $\dot{\mathbf{H}}_n$.

3.2 Constraints and costs

The transcription enforces:

$$\mathbf{v}_{n+1} - \mathbf{v}_n = h_n \dot{\mathbf{v}}_n, \qquad \mathbf{q}_{n+1} - \mathbf{q}_n = h_n\, \mathbf{B}(\mathbf{q}_n)\mathbf{v}_n, \tag{5}$$

$$m\ddot{\mathbf{c}}_n = \sum_{i=1}^{4} \mathbf{f}_{i,n} + m\mathbf{g}, \qquad \dot{\mathbf{H}}_n = \sum_{i=1}^{4} (\mathbf{p}_{i,n} - \mathbf{c}_n) \times \mathbf{f}_{i,n}, \tag{6}$$

$$\mathbf{c}_n = \mathrm{CoM}(\mathbf{q}_n), \qquad \mathbf{H}_n = \mathbf{A}(\mathbf{q}_n, \mathbf{v}_n)\mathbf{v}_n, \tag{7}$$

$$|f^x_{i,n}| \le \mu f^z_{i,n}, \qquad |f^y_{i,n}| \le \mu f^z_{i,n}, \tag{8}$$

$$0 \le f^z_{i,n} \le s_{i,n}\, 4mg, \tag{9}$$

where $s_{i,n} \in \{0, 1\}$ is the prescribed stance schedule. The kinematic constraints force a stance foot to be on the ground ($z = 0$) and not to move between consecutive stance knots, and force a swing foot to have at least 1 cm clearance. Joint position and velocity limits, unit quaternions and a body-orientation constraint are imposed at every knot. Periodic boundary conditions are imposed on the half stride (the gait is laterally symmetric, so a half stride plus a mirror reflection produces a full stride).

The cost is a quadratic running cost

$$\sum_{n=0}^{N-1} \left[(\mathbf{q}_n - \mathbf{q}_{\text{home}})^\top \mathbf{Q}_q (\mathbf{q}_n - \mathbf{q}_{\text{home}}) + \mathbf{v}_n^\top \mathbf{Q}_v \mathbf{v}_n \right], \tag{10}$$

with larger weights on the hip-roll joints (to discourage splaying), plus the implicit objective of finding any feasible gait. The problem is solved with SNOPT.

Contact-implicit versus contact-scheduled transcription

1. Choose a contact schedule: a binary stance matrix $s_{i,n}$ for each foot and knot.
2. Add contact forces only where $s_{i,n} = 1$ (normal-force upper bound $s_{i,n} 4mg$).
3. Impose the centroidal dynamics (6) and the centroidal consistency (7) at every knot.
4. Impose full kinematics: stance foot fixed on the ground, swing foot above the ground, joint limits.
5. Add periodicity and solve with a local nonlinear solver from a good initial guess (the home configuration).

3.3 Results

The notebook solves the walking-trot schedule in 8.3 s and SNOPT reports success. The optimal cost is 419.76.

Table 4: Example 5.2: walking-trot solution metrics.

Quantity	Value
Gait	walking trot (lateral symmetry)
SNOPT status	success
Optimal cost	419.76
Solve time	8.3 s
Half-stride duration	0.34375 s
Full-stride duration	0.6875 s
Stride length	0.25 m
Effective speed	0.364 m/s (requested 0.4 m/s)
CoM height range	0.1309–0.1484 m
Maximum body pitch	0.0189 rad
Maximum foot clearance	11.9 mm
Maximum normal force	74.8 N (back right)

The effective speed is slightly below the requested 0.4 m/s because the optimizer chose half-stride durations at the upper end of the allowed window ($1.1T$, with $T = 0.3125$ s). This is consistent

with the cost: the squared joint velocities penalize fast motion, so the solver stretches the stride in time until it hits the bound.

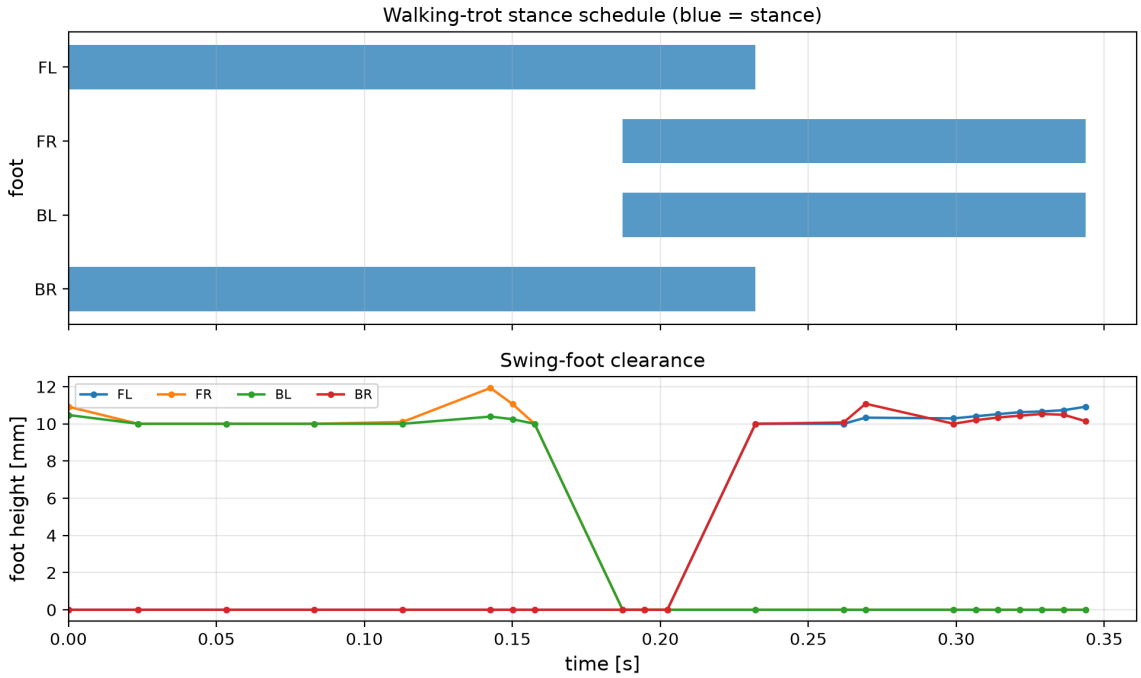


Figure 5: Example 5.2: walking-trot stance schedule and swing-foot clearance. Front-left and back-right are in stance for the first part of the half stride, then front-right and back-left take over. The swing feet clear the ground by about 10–12 mm.

What the optimizer is actually doing

The gait is not prescribed beyond the contact schedule. The optimizer has to find joint trajectories whose CoM follows the Newton–Euler equations under the four contact forces, while each stance foot stays fixed on the ground and each swing foot clears it. The angular-momentum constraint couples the contact forces to the body rotation: if the forces do not pass through the CoM, the body starts to rotate. The resulting motion is a compromise between small joint velocities, small hip-roll motion and the kinematic requirement that the feet reach the next contact.

Comments. The notebook is explicitly a work in progress: self collision is not modelled and several costs from the original MATLAB implementation are missing. The solution is therefore not necessarily dynamically feasible in a full simulation; it is a kinematic/centroidal consistency result. The value of the example is methodological: it shows how the exact centroidal dynamics, full kinematics and contact schedules fit into one nonlinear program, and it shows the role of a good initial guess (the home configuration) for a local solver. The walking trot is the easiest of the four schedules in the notebook; the running trot, rotary gallop and bound are more aggressive and are left commented out.

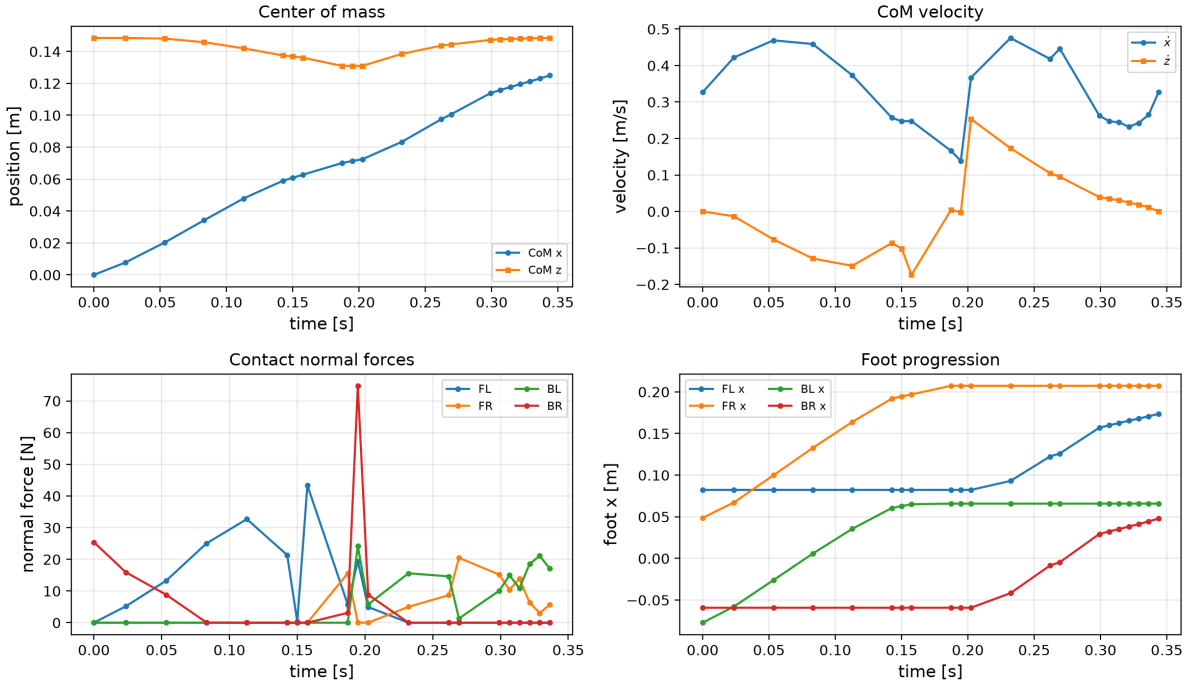


Figure 6: Example 5.2: CoM position and velocity, contact normal forces, and foot progression over one half stride. The normal forces switch between the diagonal pairs at the trot transition.

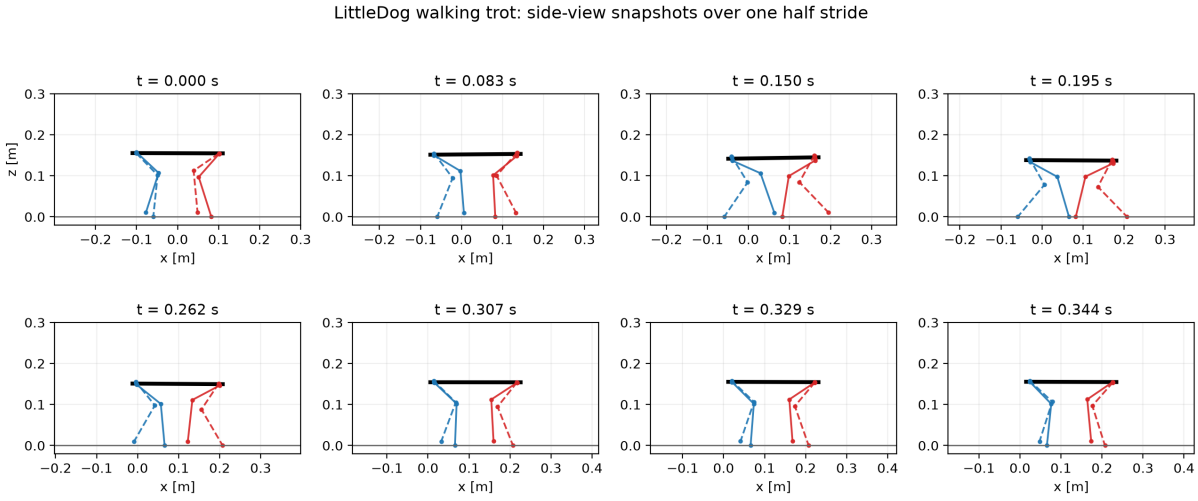


Figure 7: Example 5.2: side-view snapshots of the optimized walking trot over one half stride ($t = 0$ to 0.344 s). Solid legs are left legs, dashed legs are right legs; the body is the thick black segment. The two diagonal pairs alternate as expected.

A feasible NLP solution is not a stable gait

The optimization produces an open-loop trajectory over one half stride. Nothing in the program guarantees that the gait is stable to perturbations. Orbital stability would require a step-to-step (Poincaré) analysis, as in Chapter 4, or a feedback controller that tracks the trajectory. The periodic boundary conditions only make the trajectory repeat, not converge.

4 Section 5.8(a): Footstep Planning via MIQP

4.1 Terrain and decision variables

The terrain consists of rectangular stepping stones: an initial stone, a goal stone, a lateral stone above the bridge, and a sequence of bridge stones with a boolean presence mask. Terrain A has all four bridge stones (`[1,1,1,1]`); terrain B misses the last one (`[1,1,1,0]`).

The planner uses $n_{\text{steps}} + 1$ footsteps. The decision variables are

- $\mathbf{p}_t^L, \mathbf{p}_t^R \in \mathbb{R}^2$ for $t = 0, \ldots, n_{\text{steps}}$;
- binaries $\delta_{t,i}^L, \delta_{t,i}^R \in \{0, 1\}$, one per foot, step and stone;
- a binary `first_left` that decides which foot moves first.

4.2 The four constraints and the objective

The four pieces requested by the exercise are the following.

(a) Relative position limit. At every step each foot must lie in a square of side d centered at the other foot:

$$\left|\mathbf{p}_t^L - \mathbf{p}_t^R\right|_\infty \le \frac{d}{2}, \qquad t = 0, \ldots, n_{\text{steps}}. \tag{11}$$

(b) One stone per foot. A foot cannot be in two stones at once:

$$\sum_i \delta_{t,i}^L = 1, \qquad \sum_i \delta_{t,i}^R = 1. \tag{12}$$

(c) Foot in the assigned stone (big-M). With the halfspace representation $S_i = \{x : A_i x \le b_i\}$,

$$A_i \mathbf{p}_t^L \le b_i + (1 - \delta_{t,i}^L) M, \qquad A_i \mathbf{p}_t^R \le b_i + (1 - \delta_{t,i}^R) M, \tag{13}$$

where M is a vector of two "sufficiently large" positive constants (one per coordinate), chosen from the terrain extents. If $\delta_{t,i} = 1$ the constraint enforces membership in stone i; otherwise it is relaxed. The other binary at that step is zero by (12), so the foot cannot be simultaneously required to be in two stones.

(d) Step-length objective. Minimize the sum of squared step lengths,

$$\min \sum_{t=0}^{n_{\text{steps}}-1} \left(\|\mathbf{p}_{t+1}^L - \mathbf{p}_t^L\|^2 + \|\mathbf{p}_{t+1}^R - \mathbf{p}_t^R\|^2 \right). \tag{14}$$

The step-sequence constraint (given in the notebook) uses the binary `first_left` to allow only one foot to move per step, alternating feet; it is a big-M constraint with $M = d$ on the square centered at the previous position of the same foot.

The resulting MIQP is solved with Drake's `MixedIntegerBranchAndBound`, using OSQP for the continuous relaxations.

4.3 Results on the two benchmark terrains

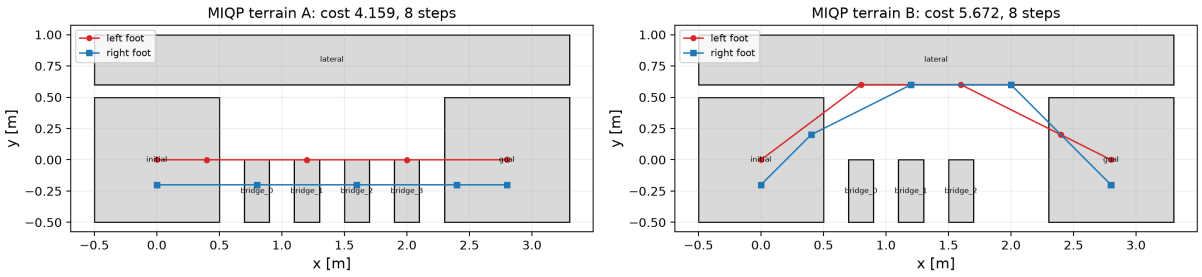


Figure 8: Section 5.8(a): optimal MIQP plans. Terrain A goes straight across the bridge; terrain B must use the lateral stone because the missing bridge stone leaves a gap that is too wide under the step-span and foot-separation limits.

Table 5: Section 5.8(a): MIQP results ($n_{\text{steps}} = 8$, $d = 0.8$ m).

Terrain	cost	solve time [s]	steps used	route	first foot
A `[1,1,1,1]`	4.1593	0.42	8	bridge	left
B `[1,1,1,0]`	5.6721	0.75	8	lateral	right

The autograder reports

Total score is 16/16: relative position 4/4, one stone per foot 3/3, foot in stone 5/5, objective 4/4.

The objective value for terrain A agrees with the reference value 4.1592510194 to 10^{-10}; for terrain B the value 5.6721 is within the autograder's 2% tolerance of 5.6479. The small difference comes from the branch-and-bound convergence tolerance, not from a modelling error.

Why the lateral stone is used in terrain B

The last bridge stone is missing. The remaining bridge ends at $x = 1.7$ m, while the goal stone starts at $x = 2.3$ m: a 0.6 m gap. With $d = 0.8$ m the feet may be at most 0.4 m apart, so a foot on the last bridge stone cannot reach the goal. The lateral stone is high enough and long enough to provide intermediate footholds; the optimizer chooses it even though the path is longer. This is exactly the kind of non-local decision that a local continuous optimizer would miss and branch and bound handles naturally.

4.4 Step-span sensitivity and route switching

We swept d from 0.6 to 1.5 m on terrain B (all other parameters fixed).

Table 6: Section 5.8(a): step-span sensitivity on terrain B.

d [m]	cost	steps used	route
0.60	–	–	infeasible
0.70	–	–	infeasible
0.75	–	–	infeasible
0.80	5.6721	8	lateral
0.90	5.5916	8	lateral
1.00	5.4091	8	lateral
1.10	5.3740	8	lateral
1.20	4.1873	8	bridge
1.30	4.1655	8	bridge
1.40	4.0317	8	bridge
1.50	4.0252	8	bridge

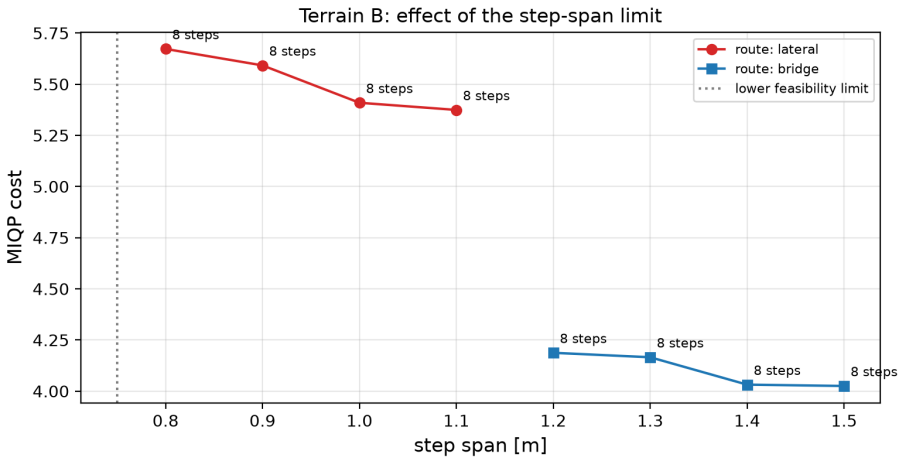


Figure 9: Section 5.8(a): cost versus step span on terrain B. The route switches from the lateral stone to the bridge between $d = 1.1$ and $d = 1.2$ m.

There are two thresholds. Below $d = 0.8$ m the problem is infeasible: the feet cannot be placed on the narrow (0.2 m wide) bridge stones while remaining within $d/2$ of each other and making

progress. Between 0.8 and 1.1 m the lateral route is feasible but the bridge is not. At $d = 1.2$ m the feet may be 0.6 m apart, which is exactly the gap from the last bridge stone to the goal; the planner switches to the shorter bridge route and the cost drops from 5.37 to 4.19. Above 1.2 m the bridge route remains optimal and the cost decreases slowly as the squared step lengths shrink.

Comments. The MIQP is a clean example of a combinatorial problem with continuous geometry. The binaries encode the discrete choice "which stone" and the big-M constraints encode the implication "if this binary is one, the foot is in this rectangle". The objective is convex quadratic, so branch and bound is guaranteed to find the global optimum. The price is exponential worst-case complexity in the number of binaries, which is why the exercise uses only four bridge stones and eight steps.

5 Section 5.8(b): Understanding the Zero-Moment Point

5.1 Free-body diagrams

We use the geometry of Fig. 10: the heel is H, the toe is T, the ankle pin is P at height h, the ankle is L_2 from the heel and L_1 from the toe, and the leg of length l carries a point mass m. The angle θ is measured from the horizontal at P.

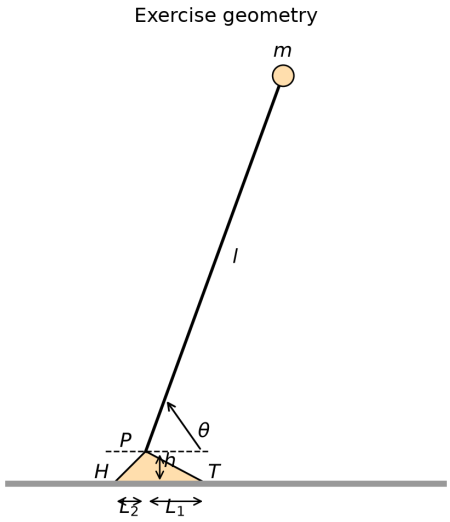


Figure 10: Geometry of the Section 5.8(b) exercise. The foot is the triangle HPT; the massless leg l carries the point mass m; θ is measured from the horizontal at the ankle P.

(a) Free-body diagrams.

- *Bob:* gravity $m\mathbf{g}$ downward and the leg force along the leg (the leg can only push or pull along its axis).
- *Leg:* massless; the forces at the two pins are equal and opposite along the leg. It is a two-force member. There is no torque at either pin.

- *Foot:* the force from the leg at P, the ground reactions at H and T (normal and friction), and no ankle torque for parts (a)–(d). Because the foot is massless, the sum of forces and the sum of moments about any point vanish.

(b) Stability at $\theta = \pi/2 + 10^{-3}$. The leg is a pendulum about P:

$$\ddot{\theta} = -\frac{g}{l}\cos\theta. \tag{15}$$

At $\theta = \pi/2 + \varepsilon$ we have $\cos\theta = -\sin\varepsilon \approx -\varepsilon$, so

$$\ddot{\theta} \approx \frac{g}{l}\varepsilon > 0. \tag{16}$$

The perturbation grows exponentially with rate $\sqrt{g/l}$ (about 3.13 s^{-1} for $l = 1$ m): the posture is *unstable*. The ZMP is at $x_{\mathrm{ZMP}} = -h\cot\theta \approx h\varepsilon$ (toward the toe), which is inside the foot for small ε; the *foot* stays flat while the *body* falls. This distinction is important: foot-flatness and postural stability are different properties.

5.2 (c) The ZMP without an ankle actuator

Because the leg is a two-force member, the force it exerts on the foot at P is along the line P–bob. The ground reaction must balance this force, so the resultant ground reaction is also along that line. The ZMP is therefore the intersection of the line P–bob with the ground. Taking P as the origin,

$$P = (0, h), \qquad \text{bob} = (l\cos\theta,\ h + l\sin\theta).$$

The line through these points hits $z = 0$ at

$$\boxed{x_{\mathrm{ZMP}}(\theta) = -h\cot\theta.} \tag{17}$$

The result is independent of $\dot{\theta}$ and of m: it is a purely geometric statement about the line of action. The equivalent derivation from the general ZMP formula

$$x_{\mathrm{ZMP}} = x_{\mathrm{cm}} - \frac{z_{\mathrm{cm}}\,\ddot{x}_{\mathrm{cm}}}{\ddot{z}_{\mathrm{cm}} + g}$$

uses $\ddot{x}_{\mathrm{cm}} = g\sin\theta\cos\theta - l\cos\theta\,\dot{\theta}^2$ and $\ddot{z}_{\mathrm{cm}} = -g\cos^2\theta - l\sin\theta\,\dot{\theta}^2$; the $\dot{\theta}$ terms cancel and give the same expression (17).

(d) Toe and heel boundaries. With the foot support $[-L_2, L_1]$ relative to P:

$$x_{\mathrm{ZMP}} = L_1 \implies \theta_T = \pi - \arctan\frac{h}{L_1}, \tag{18}$$

$$x_{\mathrm{ZMP}} = -L_2 \implies \theta_H = \arctan\frac{h}{L_2}. \tag{19}$$

At θ_T the ZMP reaches the toe. Beyond it the resultant would lie ahead of the toe, so the foot rotates about T and the *heel lifts*. At θ_H the ZMP reaches the heel; the foot rotates about H and the *toe lifts*. For the numerical values in Fig. 11 ($h = 0.08$ m, $L_1 = 0.15$ m, $L_2 = 0.08$ m) we get

$$\theta_T = 151.9^\circ, \qquad \theta_H = 45.0^\circ.$$

The counter-intuitive sign is worth emphasizing: because the leg is a strut with a frictionless ankle, leaning *backward* pushes the ZMP *forward* toward the toe, and leaning forward pushes it toward the heel.

5.3 (e)–(f) With an ideal ankle actuator

Now the ankle applies a torque that exactly cancels the gravitational torque on the leg, so $\ddot{\theta} = 0$. The robot is quasi-static and the ankle torque is internal to the robot. The only external forces are gravity at the CoM and the ground reaction; for zero moment about the CoM the ground reaction must pass through the CoM. Hence the ZMP is the vertical projection of the CoM:

$$\boxed{x_{\mathrm{ZMP}}(\theta) = l\cos\theta.} \tag{20}$$

The flat-foot condition $x_{\mathrm{ZMP}} \in [-L_2, L_1]$ gives

$$\theta \in \left[\arccos\left(\frac{L_1}{l}\right),\ \arccos\left(-\frac{L_2}{l}\right)\right], \tag{21}$$

with the arguments clipped to $[-1, 1]$. For the numerical values above this is $[81.4°, 94.6°]$, a range of only $13.2°$.

(e) About to fall. If the robot is about to tip forward, the heel normal force is zero and the ZMP is at the toe; if it is about to tip backward, the toe normal force is zero and the ZMP is at the heel. In either case one of the two ground reaction forces vanishes and the ZMP is on the boundary of the support polygon.

5.4 (g)–(h) Foot design

(g) Maximizing the posture range.

(i) *Taller or flatter?* In the static expression (20), h does not appear: the achievable range (21) depends only on L_1, L_2, l. A taller foot does not change the static range. Dynamically, however, horizontal ground forces produce a moment hF_x about the ankle; a flatter foot (smaller h) reduces the ankle torque needed for a given horizontal force, so a flatter foot is preferable in practice.

(ii) *Longer or shorter?* A longer foot (larger $L_1 + L_2$) enlarges the support polygon and therefore the range. For $L_1 = L_2 = 0.115$ m and $l = 1$ m the range is $13.2°$; for $L_1 = L_2 = 0.20$ m it is $23.1°$.

(iii) *Where to put the ankle?* For a fixed total length $L = L_1 + L_2$, the range (21) is almost independent of the ankle placement: centering gives $13.21°$ for $L = 0.23$ m, while putting the ankle at one end gives $13.28°$, a difference of $0.07°$. The total length is what matters. If a slight bias is wanted, placing the ankle at one end maximizes the total range, while centering maximizes the minimum margin on both sides. For symmetric posture robustness, center the ankle.

(h) Designing for forward walking and easy heel lift.

(i) *Taller or flatter?* The quasi-static heel-lift condition is $l\cos\theta = L_1$, which is independent of h; statically, h does not matter. Dynamically, a flatter foot reduces the ankle torque associated with horizontal push-off forces, so flatter is again preferable.

(ii) *Longer or shorter L_1?* A shorter L_1 moves the toe closer to the CoM projection and makes heel lift occur at a more upright posture. The ankle torque needed to reach the toe, $mg(L_1 - l\cos\theta)$, also decreases with L_1. So a *shorter* forefoot makes heel lift easier.

(iii) *Longer or shorter L_2?* L_2 does not enter the heel-lift condition; it sets the *backward* support margin and prevents toe lift. A longer heel does not make heel lift easier, but it improves backward stability during stance. For forward walking the useful design is a short L_1 (easy heel lift) combined with a sufficiently long L_2 (backward safety).

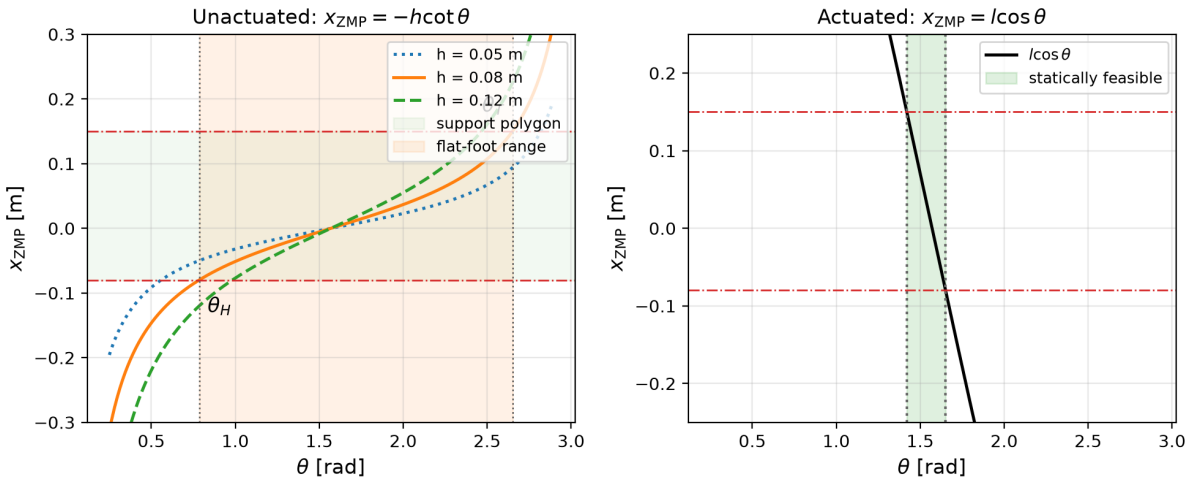


Figure 11: Section 5.8(b). Left: unactuated case, $x_{\text{ZMP}} = -h\cot\theta$ for three ankle heights; the shaded horizontal band is the support polygon and the vertical band is the flat-foot range. Right: with an ideal ankle actuator the ZMP is $l\cos\theta$; the statically feasible interval is narrow for a short foot.

Heel lift is not "the ZMP reaches the heel"

The heel lifts when the ground reaction resultant reaches the *toe*: the heel normal force becomes zero there. When the resultant reaches the *heel*, the *toe* lifts. It is easy to say this backwards. The rule is: the foot rotates about the edge that the ZMP is approaching, and the opposite edge leaves the ground.

6 Section 5.8(c): Footstep Planning via GCS

6.1 Transcription

The GCS formulation removes the binaries. Each stepping stone is a convex set, and each possible visit is a *copy* of that set, i.e. a vertex of the graph. A stone of horizontal length L gets

$$m = \left\lfloor \frac{L}{d} \right\rfloor + 2 \tag{22}$$

copies: enough variables to cross the stone in several steps and to place both feet on it. The source and target are singleton sets at the centers of the initial and goal stones.

The edges are:

- consecutive copies of the same stone: $x_{i,0} \to x_{i,1} \to \cdots$;
- from a copy of one stone to the *parent* copy ($ii = 0$) of another stone;

- from any copy of the goal stone to the target;
- from the source to the parent copy of any stone.

Each edge carries the kinematic constraint

$$\|x_u - x_v\|_\infty \le \frac{d}{2}, \tag{23}$$

which is the same foot-separation limit as (11), and a constant cost of 1 (one step). The planner then solves a shortest-path problem in the graph, with the convex relaxation of the bilinear flow constraints. The relaxation is tight for this problem: the flow variables take values 0 or 1 in the solution, and Drake's rounding step is not needed to recover the path.

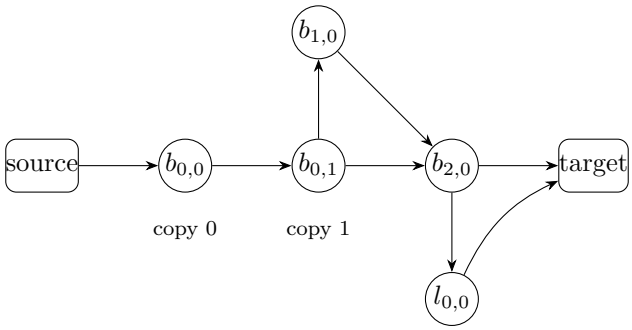


Figure 12: Schematic of the GCS for Section 5.8(c). Vertices are copies of stepping stones; edges are feasible one-step transitions. A path from `source` to `target` is a footstep plan. The copies of the same stone are chained in a fixed order.

6.2 Results

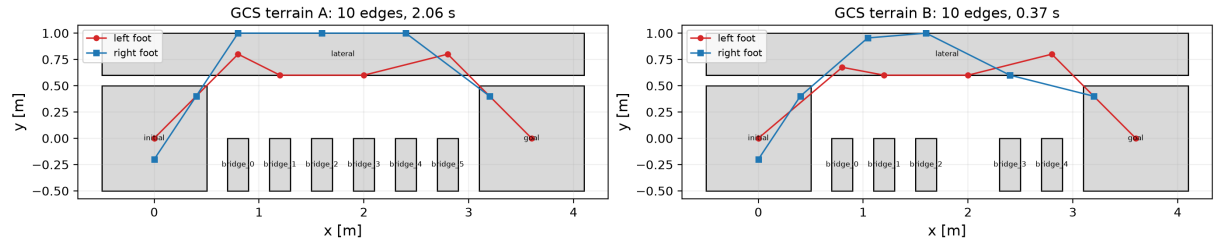


Figure 13: Section 5.8(c): GCS plans on the six-bridge-stone terrains used in the notebook. Terrain B detours through the lateral stone, as in the MIQP.

Table 7: Section 5.8(c): GCS results on the notebook terrains (`[1,1,1,1,1,1]` and `[1,1,1,0,1,1]`).

Terrain	path cost (edges)	positions	solve time [s]
A	10	11	1.02
B	10	11	0.36

The autograder reports

Total score is 16/16: relative position 8/8, objective 8/8.

The objective checked by the autograder is the squared norm of the summed displacements, which is not exactly the number-of-steps objective used in the notebook; the solution passes both because the paths use the same number of edges and the same geometric route.

6.3 MIQP versus GCS

Table 8: Solving the same bridge terrain with MIQP and GCS. The MIQP uses $n_{\text{steps}} = n_{\text{bridge}} + 5$; the GCS has no step limit. Times are single-run wall-clock values and exclude model construction.

Bridge stones	MIQP time [s]	GCS time [s]	GCS path edges
4	0.96	0.39	7
6	2.23	0.44	10
8	5.00	0.77	11

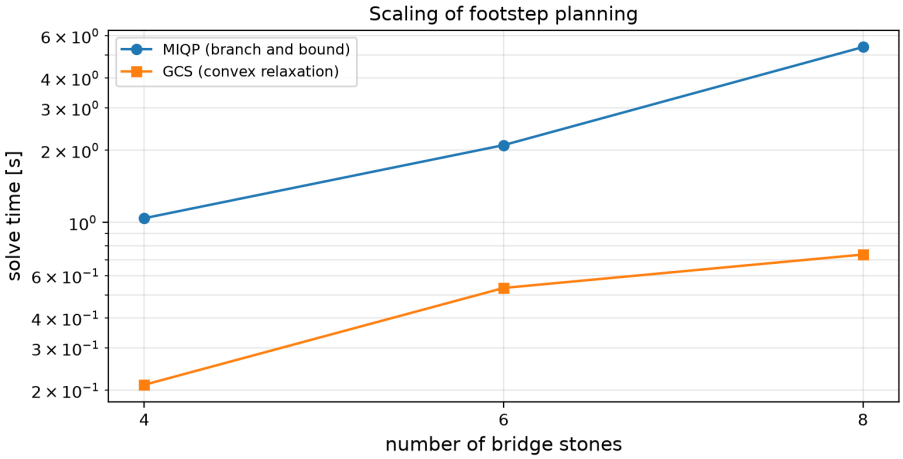


Figure 14: Section 5.8(c): solve time versus terrain size. The MIQP grows much faster than the GCS convex relaxation on this range.

Why GCS is faster

The MIQP has one binary per foot, step and stone; branch and bound must explore combinations of them. The GCS encodes the same discrete choice as a shortest path, and the convex relaxation of the flow variables is a linear program. For footstep planning the relaxation is unusually tight, so the relaxed solution is already integral and no branch and bound is needed. The price is that the GCS as implemented here minimizes the number of steps, not the squared step length; the two objectives coincide on the benchmark terrains but not in general.

Comments. The GCS is the more scalable formulation, but it requires careful graph construction: the number of copies, the parent-only entry rule, and the fixed order of copies. A naive complete graph would be correct but much larger; the rules in the exercise keep it sparse. The MIQP is the more transparent formulation for teaching because each constraint has a direct logical meaning. In practice, the choice depends on the objective and on the available solver: MIQP with a commercial solver for quadratic step-length costs, GCS for very large terrains and linear costs.

7 Conclusions

1. **ZMP planning.** The LIPM turns the CoM/ZMP relation into a linear control problem. Drake's `ZmpPlanner` returns both a nominal CoM trajectory and a feedback policy. The policy rejects a 0.30 m/s push and a 5 cm CoM offset within one 8.5 s footstep plan. The timing study shows the expected trade-off: longer single support means more lateral sway, while longer double support means a longer plan but smoother ZMP tracking.

2. **LittleDog.** The gait optimization is a direct transcription with exact centroidal dynamics, full kinematics, contact forces and a prescribed contact schedule. SNOPT finds a walking trot in 8.3 s with a full stride of 0.6875 s, a 0.25 m stride and a peak foot clearance of 11.9 mm. The result is a kinematic/centroidal plan, not a proof of orbital stability.

3. **MIQP footstep planning.** The four big-M components (relative position, one stone per foot, foot-in-stone, squared step length) solve the benchmark terrains to the global optimum and pass 16/16. The step-span sweep shows two thresholds: feasibility at $d = 0.8$ m and the lateral-to-bridge route switch at $d = 1.2$ m.

4. **The ZMP exercise.** Without an ankle actuator the leg is a two-force member and the ZMP is the intersection of the leg line with the ground, $x_{\mathrm{ZMP}} = -h \cot\theta$. Heel lift occurs when the ZMP reaches the toe, $\theta_T = \pi - \arctan(h/L_1)$. With an ideal ankle actuator the ZMP is the CoM projection, $x_{\mathrm{ZMP}} = l\cos\theta$, and the posture range is narrow for a short foot. The total foot length dominates the design; h cancels statically.

5. **GCS footstep planning.** The graph-of-convex-sets transcription replaces branch and bound with a shortest path and a tight convex relaxation. It passes 16/16 and scales better than the MIQP on the tested terrains (0.77 s versus 5.0 s for eight bridge stones).

8 Code and Reproduction

The complete code is in the submission. Main entry points:

Topic	Script / notebook
Example 5.1	`Exp5'1-zmp-planner.ipynb`, `zmp_analysis.py`
Example 5.2	`Exp5'2-littledog.ipynb`, `littledog_analysis.py`
Section 5.8(a)	`Exp5'8-footstep-planning.ipynb`, `footstep_analysis.py`
Section 5.8(b)	`zmp_exercise_analysis.py`
Section 5.8(c)	`Exp5'8-footstep-planning-gcs.ipynb`, `footstep_analysis.py`

Commands (from `Ch-5/exp5_analysis`):

```
/Users/leo/venvs/drake/bin/python zmp_analysis.py
/Users/leo/venvs/drake/bin/python littledog_analysis.py
/Users/leo/venvs/drake/bin/python footstep_analysis.py
/Users/leo/venvs/drake/bin/python zmp_exercise_analysis.py
```

Headless notebook execution:

```
COLAB_TESTING=1 /Users/leo/venvs/drake/bin/jupyter nbconvert \
  --to notebook --execute --inplace --ExecutePreprocessor.timeout=1200 \
  "Exp5'1-zmp-planner.ipynb"
# The LittleDog notebook must run without COLAB_TESTING so that
# running_as_notebook is true and SNOPT gets its full iteration budget.
/Users/leo/venvs/drake/bin/jupyter nbconvert --to notebook --execute \
  --inplace --ExecutePreprocessor.timeout=3000 "Exp5'2-littledog.ipynb"
```

The report is compiled with

```
pdflatex Ch5_Report.tex
pdflatex Ch5_Report.tex
```

A Key Code Snippets

A.1 ZMP planner evaluation

```python
def plan_walk(footsteps, double_support_duration, single_support_duration,
              height=1.0, x0=None):
    if x0 is None:
        x0 = np.zeros(4)
    zmp_traj = GenerateDesiredZmpTraj(
        footsteps, double_support_duration, single_support_duration)
    zmp_planner = ZmpPlanner()
    zmp_planner.Plan(zmp_traj, x0, height)
    return zmp_planner, zmp_traj

# Closed-loop rollout with the feedback policy:
for t in ts:
    c = planner.get_nominal_com(t)
    cd = planner.get_nominal_comd(t)
    x = np.concatenate((c, cd))
    cdd = planner.ComputeOptimalCoMdd(t, x)
    zmp = planner.comdd_to_cop(x, cdd)
```

A.2 MIQP constraints

```
# (a) relative position
for t in range(n_steps + 1):
    relative = position_left[t] - position_right[t]
    prog.AddLinearConstraint(le(relative, step_span / 2))
    prog.AddLinearConstraint(ge(relative, -step_span / 2))

# (b) one stone per foot
for t in range(n_steps + 1):
    prog.AddLinearConstraint(stone_left[t].sum() == 1)
    prog.AddLinearConstraint(stone_right[t].sum() == 1)

# (c) big-M foot-in-stone
for t in range(n_steps + 1):
    for i, stone in enumerate(terrain.stepping_stones):
        prog.AddLinearConstraint(le(
            stone.A.dot(position_left[t]),
            stone.b + (1 - stone_left[t, i]) * M))
        prog.AddLinearConstraint(le(
            stone.A.dot(position_right[t]),
            stone.b + (1 - stone_right[t, i]) * M))

# (d) squared step length
for t in range(n_steps):
    dl = position_left[t + 1] - position_left[t]
    dr = position_right[t + 1] - position_right[t]
    prog.AddQuadraticCost(dl.dot(dl))
    prog.AddQuadraticCost(dr.dot(dr))
```

A.3 GCS edges

```
def add_vertex_and_edges_for_set_copies(G, T, step_span):
    ...
    def num_copies(stone):
        return int(np.floor(stone.width / step_span)) + 2

def add_edges_towards_parent_vertices(G, V, E_list):
    for u in V:
        if u.name() == "target":
            continue
        u_stem, _ = split_name(u.name())
        for v in V:
            if v.name() == "source":
                continue
            v_stem, v_idx = split_name(v.name())
            if u.name() == "source":
                if v.name() != "target" and v_idx == 0:
                    E_list.append([u, v])
            elif v.name() == "target":
                E_list.append([u, v])
            elif v_stem != u_stem and v_idx == 0:
                E_list.append([u, v])
```

```
def add_edge_constraints_costs(G, E_list, step_span):
    for u, v in E_list:
        e = G.AddEdge(u, v)
        for k in range(2):
            e.AddConstraint(e.xu()[k] - e.xv()[k] <= step_span / 2)
            e.AddConstraint(e.xv()[k] - e.xu()[k] <= step_span / 2)
        e.AddCost(1.0)
```

A.4 LittleDog centroidal constraints (sketch)

```
# CoM Newton-Euler
for n in range(N - 1):
    prog.AddConstraint(eq(com[:, n + 1], com[:, n] + h[n] * comdot[:, n]))
    prog.AddConstraint(eq(comdot[:, n + 1], comdot[:, n] + h[n] * comddot[:, n]))
    prog.AddConstraint(eq(
        total_mass * comddot[:, n],
        sum(contact_force[i][:, n] for i in range(4))
        + total_mass * gravity))

# Angular momentum about the CoM
for n in range(N - 1):
    prog.AddConstraint(eq(H[:, n + 1], H[:, n] + h[n] * Hdot[:, n]))
    prog.AddConstraint(partial(angular_momentum_constraint,
                               context_index=n),
                       lb=np.zeros(3), ub=np.zeros(3),
                       vars=np.concatenate((q[:, n], com[:, n],
                                            Hdot[:, n], Fn)))

# Centroidal consistency: com == CenterOfMass(q),
# H == SpatialMomentumInWorldAboutPoint(q, v, com)
for n in range(N):
    prog.AddConstraint(partial(com_constraint, context_index=n),
                       lb=np.zeros(6), ub=np.zeros(6),
                       vars=np.concatenate((q[:, n], v[:, n],
                                            com[:, n], H[:, n])))
```